

# Playing Atari with Deep Reinforcement Learning

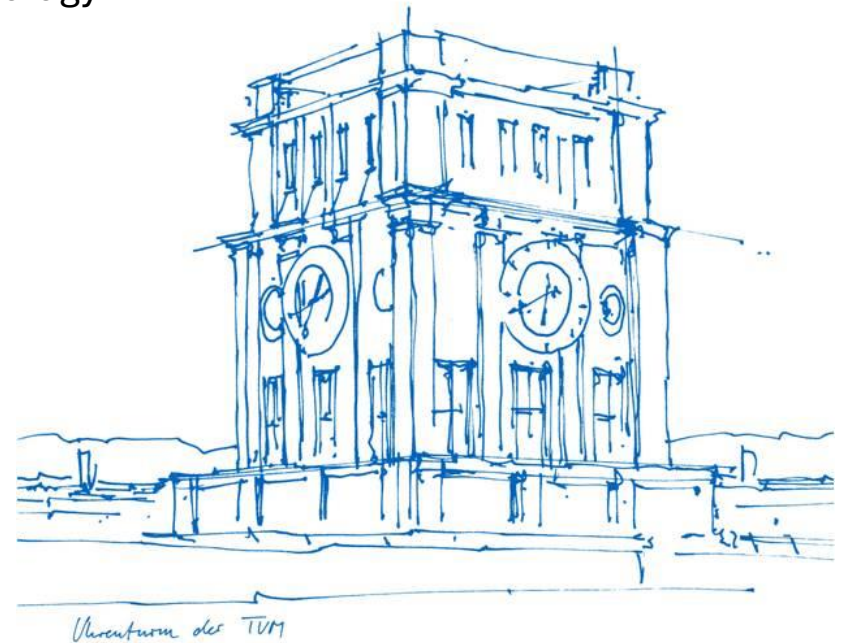
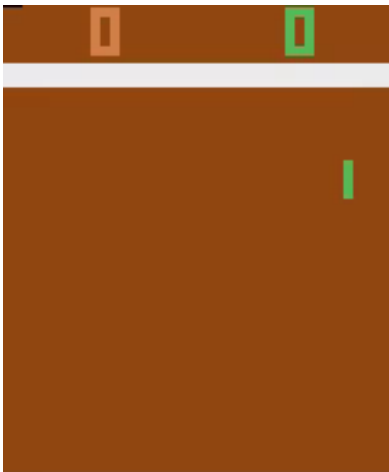
Paul Vorderbrügge, Sebastian Rogg, Luca Bozzetti

Technische Universität München

TUM School of Computation, Information and Technology

Seminar - Decision Making Algorithms

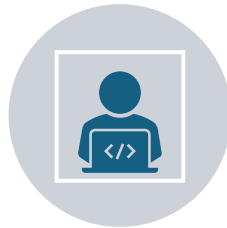
09. Juli 2025



# Outline



FOUNDATIONS



ALGORITHM



CODE



RESULTS &  
OUTLOOK

# Overview

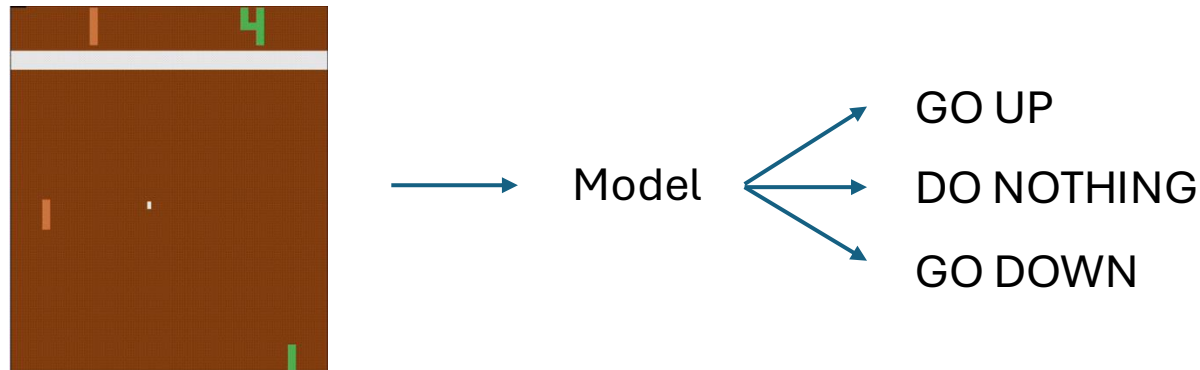


Figure 1: Screen shots from five Atari 2600 Games: (Left-to-right) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

Source: Mnih et al. 2013

- **First deep learning model** to learn control policies **directly** from high-dimensional **sensory input** (i.e. images) using reinforcement learning.
- Prior to this work: hand-engineered features, incorporating large prior knowledge about the problem with simple (e.g. linear) value functions

# Motivation - Why Raw Input?



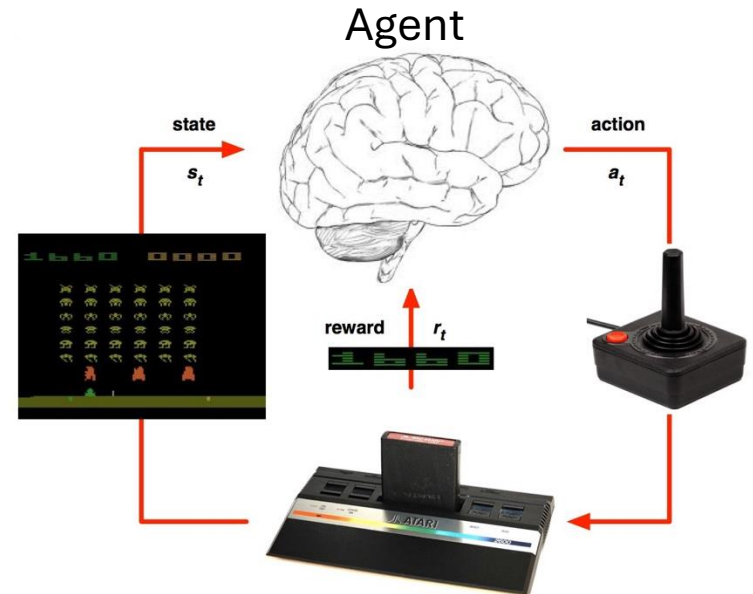
- Model doesn't need prior knowledge of the problem
- Models' performance isn't reliant on the quality of a hand-crafted feature
- Easily applicable to different problems (the same model used for all games)
- Learns just like a human

# The RL Problem

At each time step  $t$ :

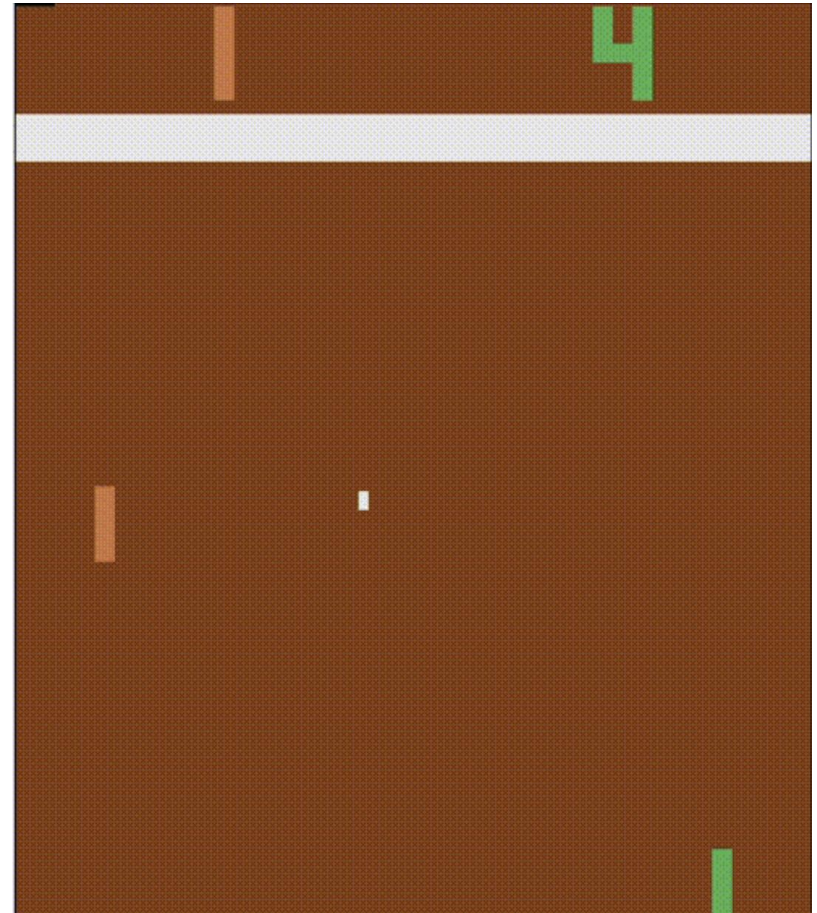
- Agent chooses action  $a_t$  from a set of legal actions  $A = \{1, \dots, K\}$
- Atari Emulator runs the action and returns next state  $s_{t+1}$  and reward  $r_t$
- Goal: select actions to maximise (discounted) future reward defined as

$$R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$$



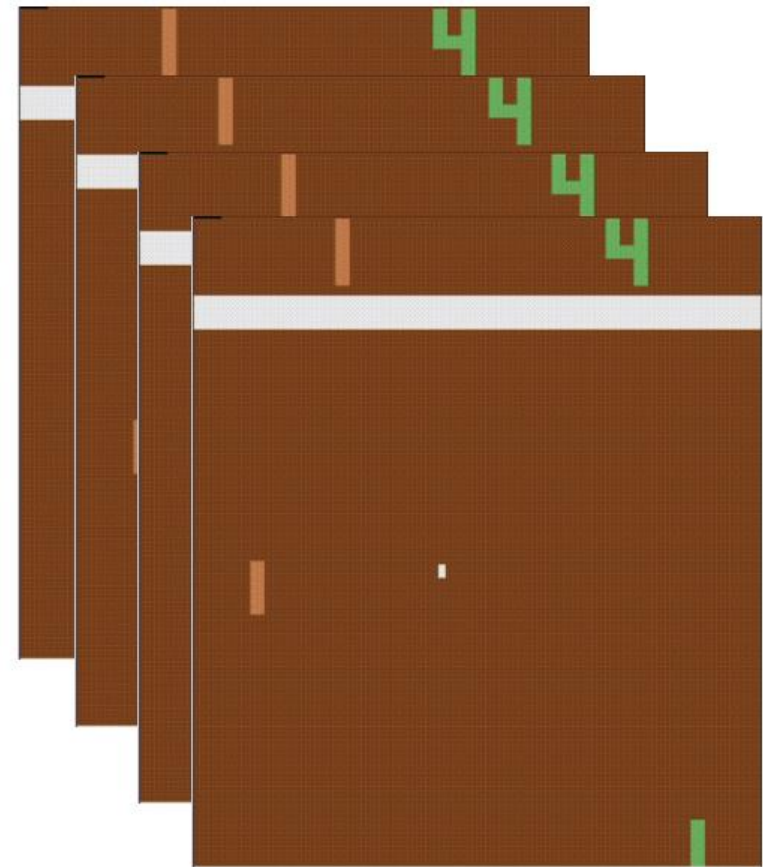
# State

- First idea: just use a frame as a state
- Problem: Where is the ball going?



# State

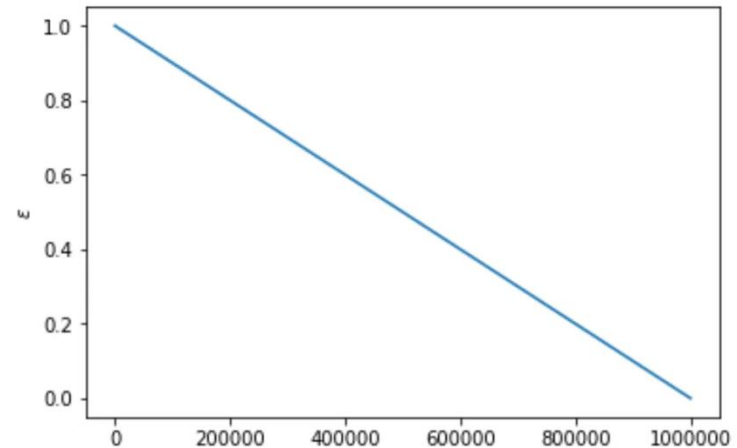
- Stack 4 frames as one state
- This accounts for velocity



# Exploration vs. Exploitation

## $\epsilon$ -greedy strategy

- Execute a random action with probability  $\epsilon$
- Execute the greedy (best) action with probability  $1 - \epsilon$
- In practice  $\epsilon$  is reduced over time





# Q-Learning

- Old concept (1989) but not successfully combined with DL before
- Q-function maps {state, action} pairs to future reward:

$$Q : S \times A \rightarrow \mathbb{R}$$

- Optimal q-function maps {state, action} pairs to the max expected reward achievable by any policy:

$$Q^*(s, a) = \mathbb{E}_{s'}[r + \gamma \max_{a'} Q^*(s', a') | s, a]$$

# Q-Learning

- Can't use Q-table for Q-values
- Instead use a neural network to approximate the correct Q-values
- Q-function gets parameter  $\theta$  (weights of the neural network)

$$Q^*(s, a) \approx Q(s, a; \theta)$$

|          | $A_1$         | $A_2$         | ...      | $A_M$         |
|----------|---------------|---------------|----------|---------------|
| $s_1$    | $Q(s_1, A_1)$ | $Q(s_1, A_2)$ |          | $Q(s_1, A_M)$ |
| $s_2$    | $Q(s_2, A_1)$ | $Q(s_2, A_2)$ |          | $Q(s_2, A_M)$ |
| $\vdots$ |               |               | $\ddots$ | $\vdots$      |
| $s_N$    | $Q(s_N, A_1)$ | $Q(s_N, A_2)$ | ...      | $Q(s_N, A_M)$ |

# Approximate $Q^*$ with a NN

Training:

Q-Network is trained by minimising a loss function:

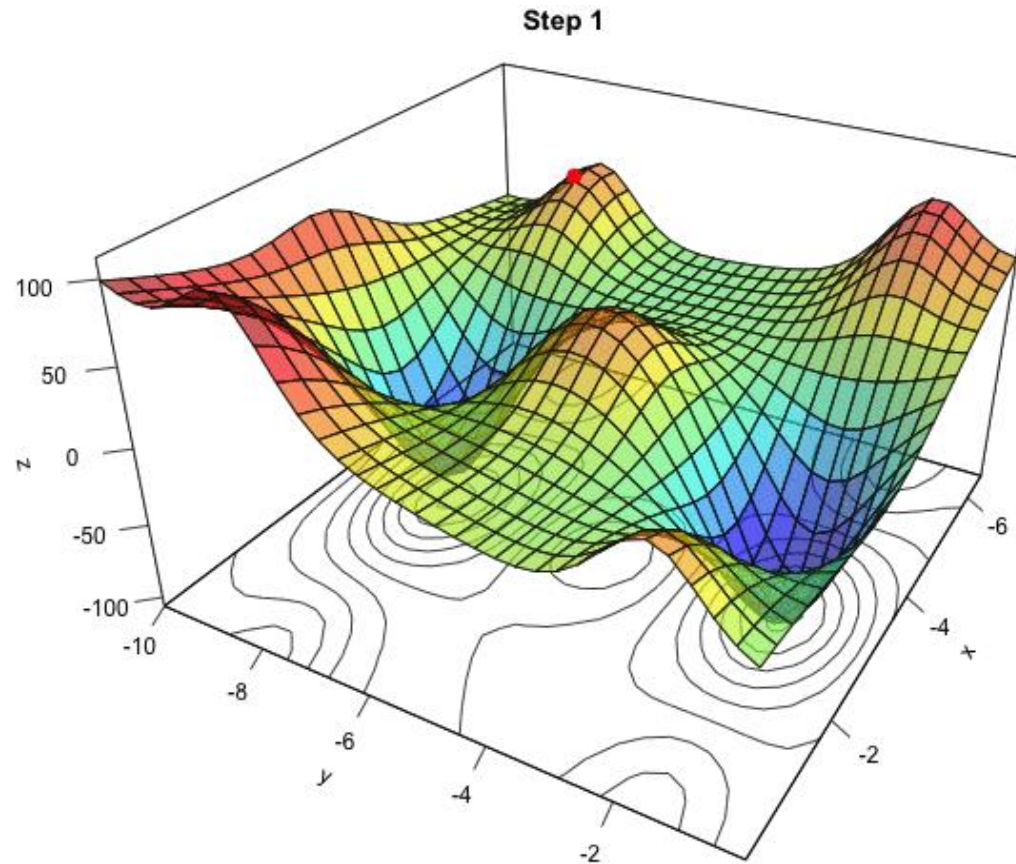
$$L_i(\theta_i) = \mathbb{E}_{s,a \sim p}[(y_i - Q(s, a; \theta_i))^2]$$

with the target defined as:

$$y_i = \mathbb{E}_{s'}[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) | s, a]$$

through gradient descent

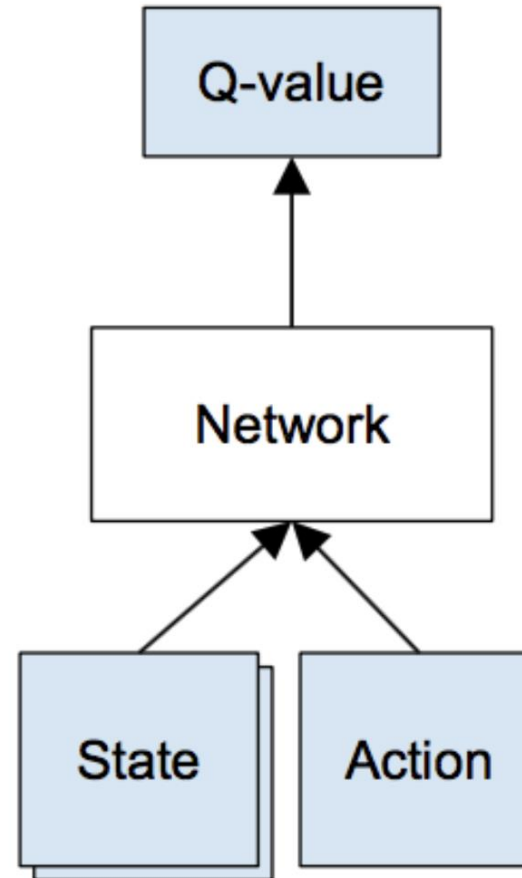
# Gradient Descent



# Deep Q-Learning

Naive idea:

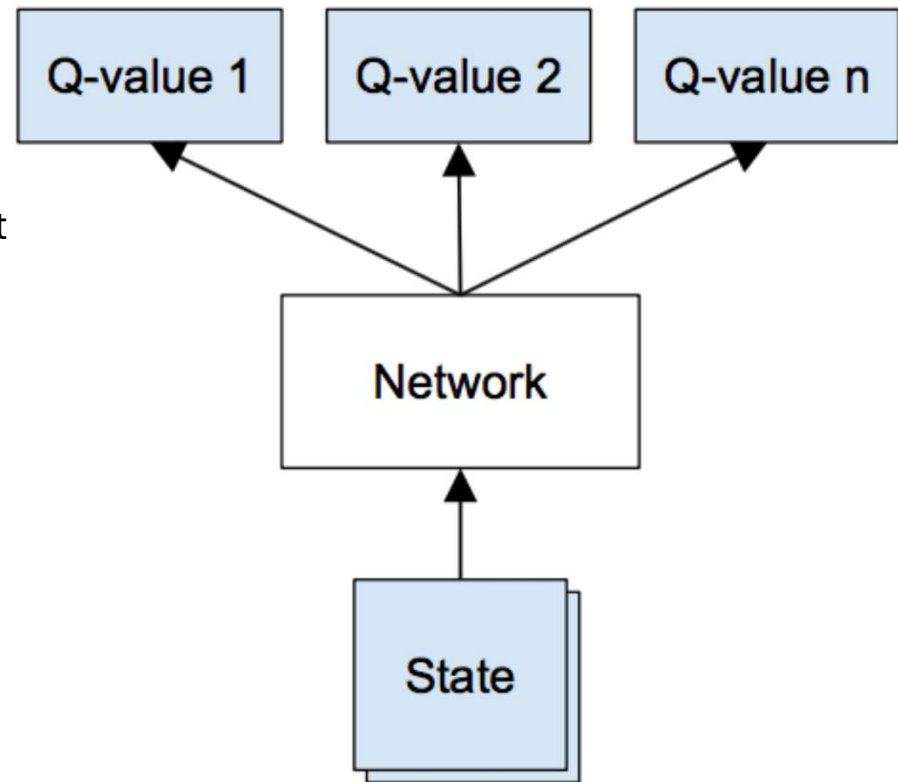
1. Load **state and action** into neural network get its Q-value – repeat for all actions
2. Execute action with best Q-value and get reward
3. Train network based on gathered information  $\langle s, a, r, s' \rangle$



# Deep Q-Learning

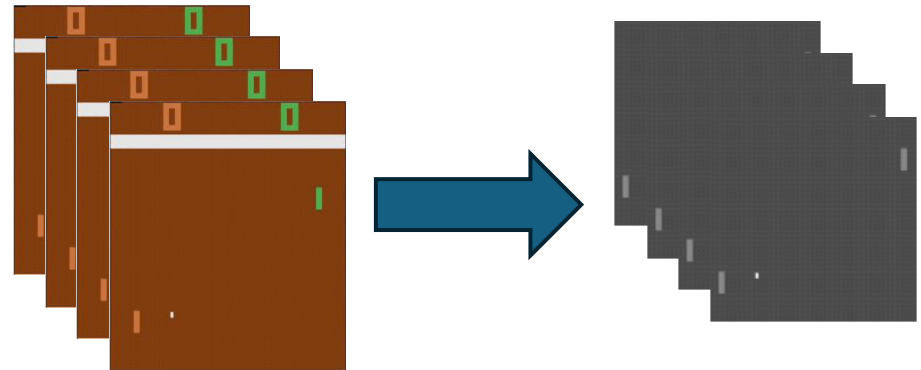
Better idea:

1. Load **state** into neural network get vector of Q-values
2. Execute action with best Q-value and get reward
3. Train network based on gathered information  $\langle s, a, r, s' \rangle$



# State

- Working with 210x160 px and 128 RGB palette is demanding
- Grayscale
- Down-sampling 110x84 px
- Crop 84x84 px
- $\phi_1 = \phi(s_1)$



# Training - feed in states and learn?

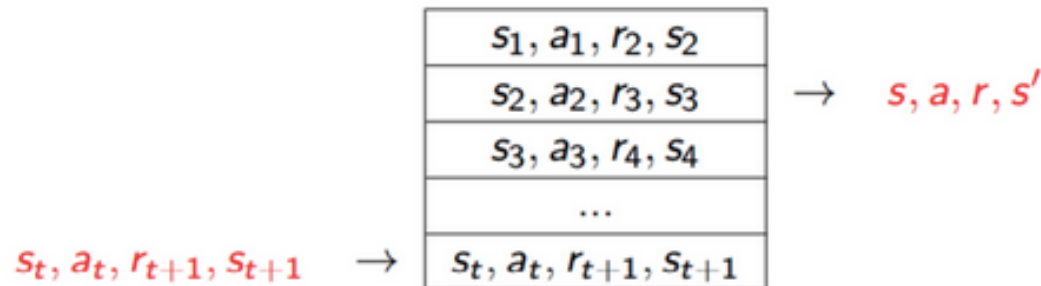
- DL assumes data samples are **i.i.d.**
- In RL: sequence of highly **correlated** states;  
data distribution changes as policy changes





# Training - feed in states and learn?

- Solution: Experience replay in a replay memory  $D$  (Long-Ji Lin, 1993)
  - Store agent's experiences  $\langle s, a, r, s' \rangle$
  - Random sampling **breaks correlation**
  - Update  $D$



# **Alg. 1: Deep Q-learning with Experience Replay**

# Alg. 1: Deep Q-learning with Experience Replay

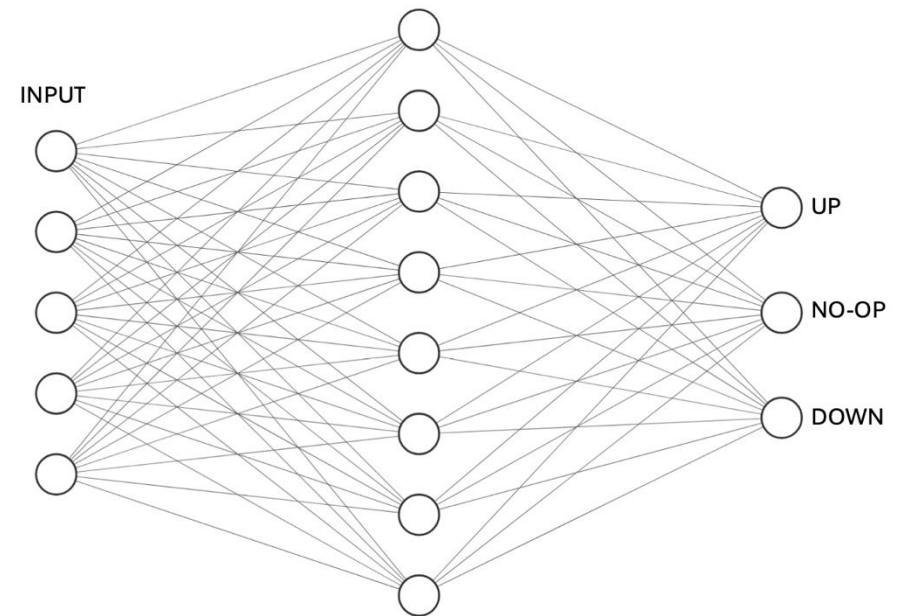
→ Initialize replay memory  $\mathcal{D}$  to capacity  $N$

| Replay Memory |
|---------------|
| 1             |
| ...           |
| N             |

# Alg. 1: Deep Q-learning with Experience Replay

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

→ Initialize action-value function  $Q$  with random weights



# Alg. 1: Deep Q-learning with Experience Replay

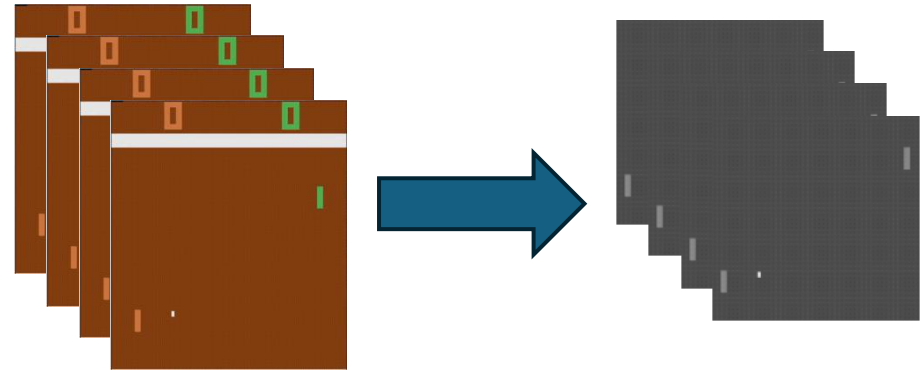
Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

→ Initialize sequence  $s_1 = \{x_1\}$

→ Preprocess sequence  $\phi_1 = \phi(s_1)$



# Alg. 1: Deep Q-learning with Experience Replay

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

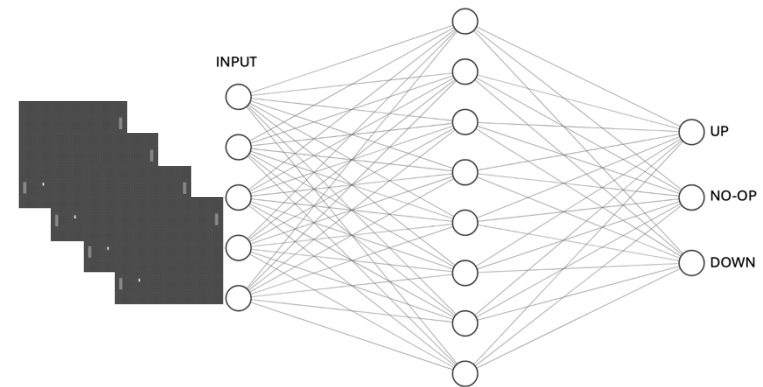
Initialize sequence  $s_1 = \{x_1\}$

Preprocess sequence  $\phi_1 = \phi(s_1)$

**for**  $t = 1, T$  **do**

With probability  $\varepsilon$  select a random action  $a_t$

Otherwise select  $a_t = \operatorname{argmax}_a Q^*(\phi(s_t), a; \theta)$



# Alg. 1: Deep Q-learning with Experience Replay

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

    Initialize sequence  $s_1 = \{x_1\}$

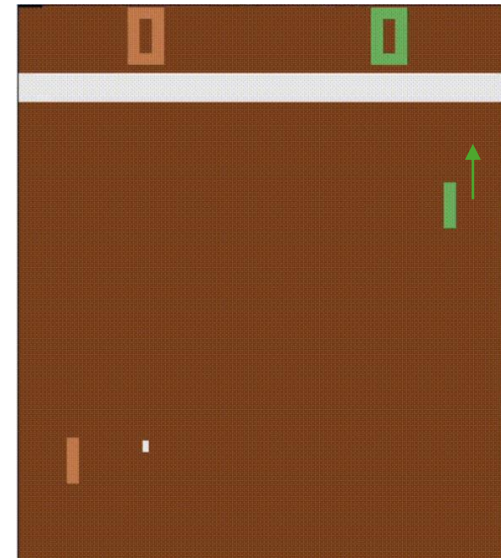
    Preprocess sequence  $\phi_1 = \phi(s_1)$

**for**  $t = 1, T$  **do**

        With probability  $\varepsilon$  select a random action  $a_t$

        Otherwise select  $a_t = \operatorname{argmax}_a Q^*(\phi(s_t), a; \theta)$

        Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$



# Alg. 1: Deep Q-learning with Experience Replay

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

    Initialize sequence  $s_1 = \{x_1\}$

    Preprocess sequence  $\phi_1 = \phi(s_1)$

**for**  $t = 1, T$  **do**

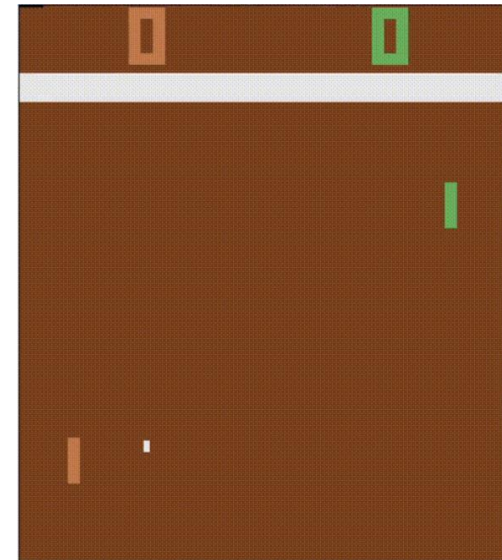
        With probability  $\varepsilon$  select a random action  $a_t$

        Otherwise select  $a_t = \operatorname{argmax}_a Q^*(\phi(s_t), a; \theta)$

        Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$

→ Set  $s_{t+1} = s_t a_t x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$

→ Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$



| Replay Memory           |
|-------------------------|
| $(s_t, UP, 0, s_{t+1})$ |
|                         |
|                         |



# Alg. 1: Deep Q-learning with Experience Replay

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

Initialize sequence  $s_1 = \{x_1\}$

Preprocess sequence  $\phi_1 = \phi(s_1)$

**for**  $t = 1, T$  **do**

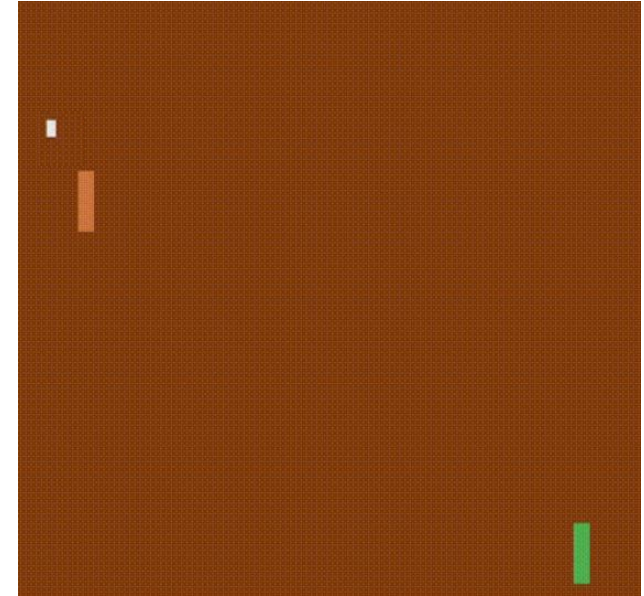
With probability  $\varepsilon$  select a random action  $a_t$

Otherwise select  $a_t = \operatorname{argmax}_a Q^*(\phi(s_t), a; \theta)$

Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$

→ Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$

→ Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$



Replay Memory

$(s_1, UP, 0, s_2)$

$(s_2, DOWN, 0, s_3)$

→  $(s_t, UP, 1, s_{t+1})$

# Alg. 1: Deep Q-learning with Experience Replay

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

Initialize sequence  $s_1 = \{x_1\}$

Preprocess sequence  $\phi_1 = \phi(s_1)$

**for**  $t = 1, T$  **do**

With probability  $\varepsilon$  select a random action  $a_t$

Otherwise select  $a_t = \operatorname{argmax}_a Q^*(\phi(s_t), a; \theta)$

Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$

Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$

Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$

→ Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$

→ Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

→ perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$

**end for**

**end for**

| Replay Memory           |
|-------------------------|
| $(s_1, UP, 0, s_2)$     |
| $(s_2, DOWN, 0, s_3)$   |
| $(s_t, UP, 1, s_{t+1})$ |

# Alg. 1: Deep Q-learning with Experience Replay

Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$

→ Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

→ perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$

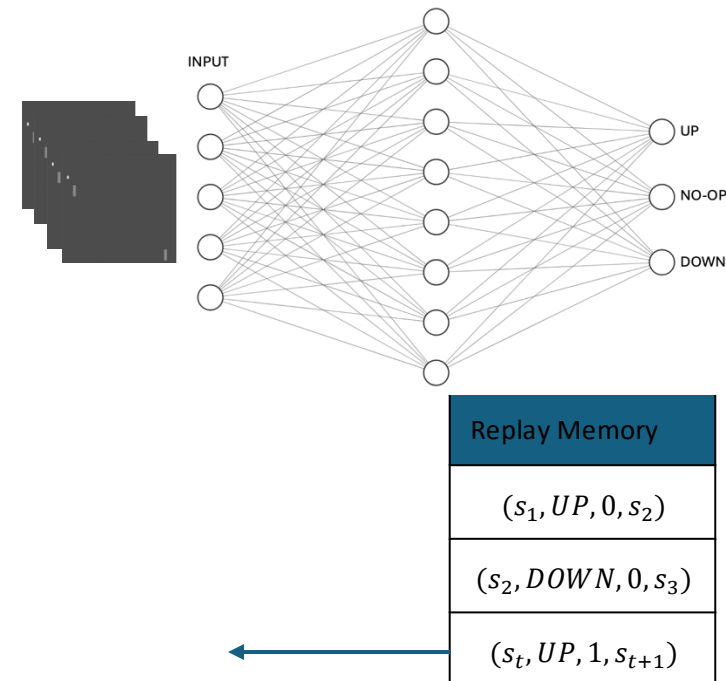
## Example

$$y_j = 1$$

$$(1 - Q(\phi_t, UP; \theta))^2$$

perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$

$Q(s_t, UP)$  gets closer to 1



# Alg. 1: Deep Q-learning with Experience Replay

Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$

→ Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

→ perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$

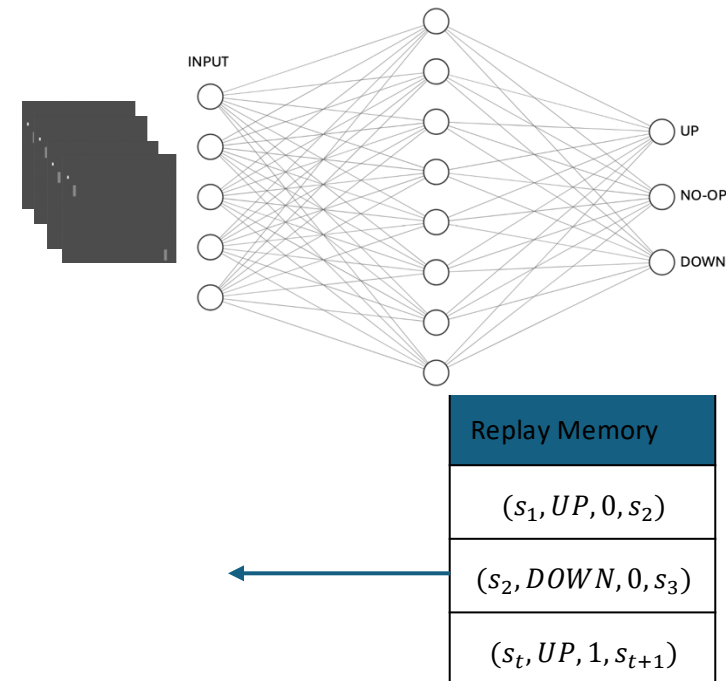
## Example 2

$$y_j = 0 + \max_{a'} Q(\phi_3, a'; \theta)$$

$$(y_j - Q(\phi_2, \text{DOWN}; \theta))^2$$

perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$

$Q(s_2, \text{DOWN})$  gets closer to  $y_j$



# Alg. 1: Deep Q-learning with Experience Replay

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

Initialize sequence  $s_1 = \{x_1\}$

Preprocess sequence  $\phi_1 = \phi(s_1)$

**for**  $t = 1, T$  **do**

With probability  $\varepsilon$  select a random action  $a_t$

Otherwise select  $a_t = \operatorname{argmax}_a Q^*(\phi(s_t), a; \theta)$

Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$

Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$

Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$

Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$

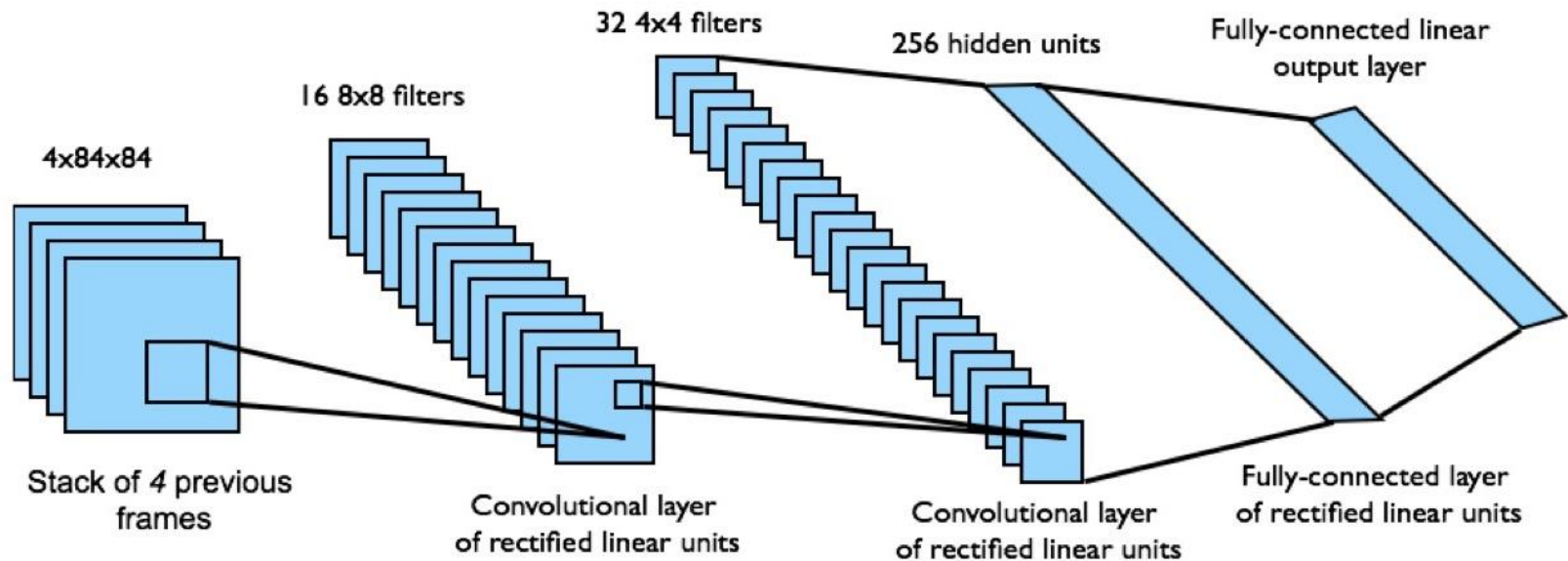
Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$

**end for**

**end for**

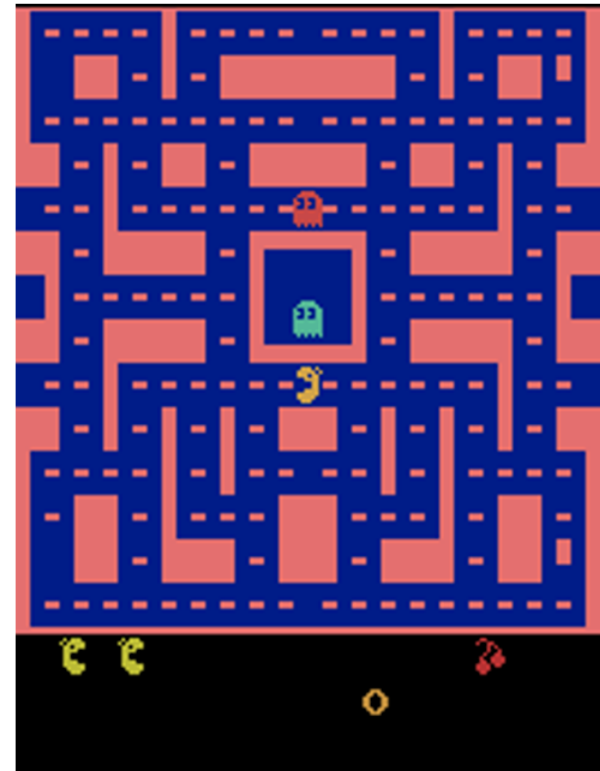
# DEEP Q-NETWORK ARCHITECTURE



# Gymnasium Atari Environments (ALE)



Pong



Ms PacMan

# Convolutional Neural Network

```
Convolutional Neural Network

1 class OriginalConvNet(nn.Module):
2     """ The neural network used to predict Q values """
3     def __init__(self, input_frames, output_dim):
4         """ output_dim corresponds to the number of possible actions """
5         super().__init__()
6
7         # Convolutional Layers
8         self.conv = nn.Sequential(
9             nn.Conv2d(input_frames, 16, kernel_size=8, stride=4), # output shape: 20 x 20 x 16
10            nn.ReLU(),
11            nn.Conv2d(16, 32, kernel_size=4, stride=2), # output shape: 9 x 9 x 32
12            nn.ReLU(),
13        )
14
15        # Fully Connected Layers
16        self.fc = nn.Sequential(
17            nn.Linear(32 * 9 * 9, 256),
18            nn.ReLU(),
19            nn.Linear(256, output_dim)
20        )
```



# Initialization and Loops

```
1 def train(self):
2     frames_trained = 0
3     episode_num = 0
4     while frames_trained < TOTAL_FRAMES: # for episode = 1, M do
5         state = self.atari.reset()
6         episode_reward = 0.
7         while True: # for t = 1, T do
8             frames_trained += 1
```

# Epsilon-Greedy Strategy



```
1 # With probability epsilon select a random action at
2 if random.random() < self.epsilon.get_and_update():
3     action = random.randrange(self.atari.amount_actions)
4 else:
5     action = self.model.act(state)|
```

# Execute action and store in replay memory

```
1 # Execute action a_t in emulator and observe reward rt and image xt+1
2 next_state, reward, done = self.atari.step(action)
3
4 # Store transition ( $\phi_t$ ,  $a_t$ ,  $r_t$ ,  $\phi_{t+1}$ ) in D (memory)
5 self.memory.push(state, action, reward, next_state, done)
6 state = next_state
7 episode_reward += reward
```

# Sample Transaction and perform Gradient Descent

```
1 # Sample random batch and compute Loss
2 batch = self.memory.sample(self.batch_size)
3 q_values = self.q(batch)
4 y = self.compute_y(batch)
5 loss = (y - q_values).pow(2).mean()
6
7 # Perform gradient descent step
8 self.gradient_descent(loss)
```

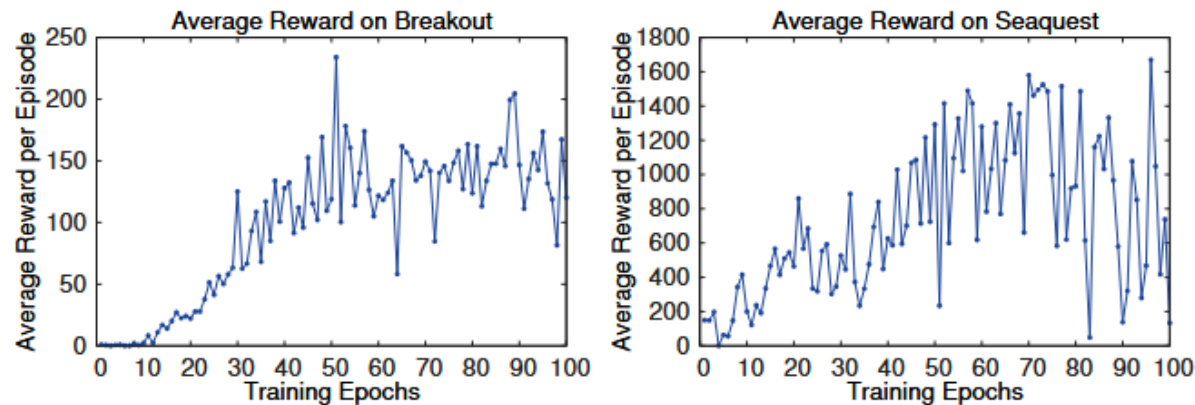
# How to compute Y (Optimal Q Prediction)

```
1 # run a forward pass through the model with the next state s'
2 q_values_new = self.q(next_state)
3
4 # select the highest q value
5 q_value_new = q_values_new.max(1)[0]
6
7 #  $r_j + \gamma * \max_{a'} Q(\phi_{j+1}, a'; \theta)$ 
8 y = reward + self.gamma * q_value_new * (1 - done)
```

# Results

|                 | B. Rider    | Breakout   | Enduro     | Pong      | Q*bert      | Seaquest    | S. Invaders |
|-----------------|-------------|------------|------------|-----------|-------------|-------------|-------------|
| Random          | 354         | 1.2        | 0          | −20.4     | 157         | 110         | 179         |
| Sarsa [3]       | 996         | 5.2        | 129        | −19       | 614         | 665         | 271         |
| Contingency [4] | 1743        | 6          | 159        | −17       | 960         | 723         | 268         |
| DQN             | <b>4092</b> | <b>168</b> | <b>470</b> | <b>20</b> | <b>1952</b> | <b>1705</b> | <b>581</b>  |
| Human           | 7456        | 31         | 368        | −3        | 18900       | 28010       | 3690        |
| HNeat Best [8]  | 3616        | 52         | 106        | 19        | 1800        | 920         | <b>1720</b> |
| HNeat Pixel [8] | 1332        | 4          | 91         | −16       | 1325        | 800         | 1145        |
| DQN Best        | <b>5184</b> | <b>225</b> | <b>661</b> | <b>21</b> | <b>4500</b> | <b>1740</b> | 1075        |

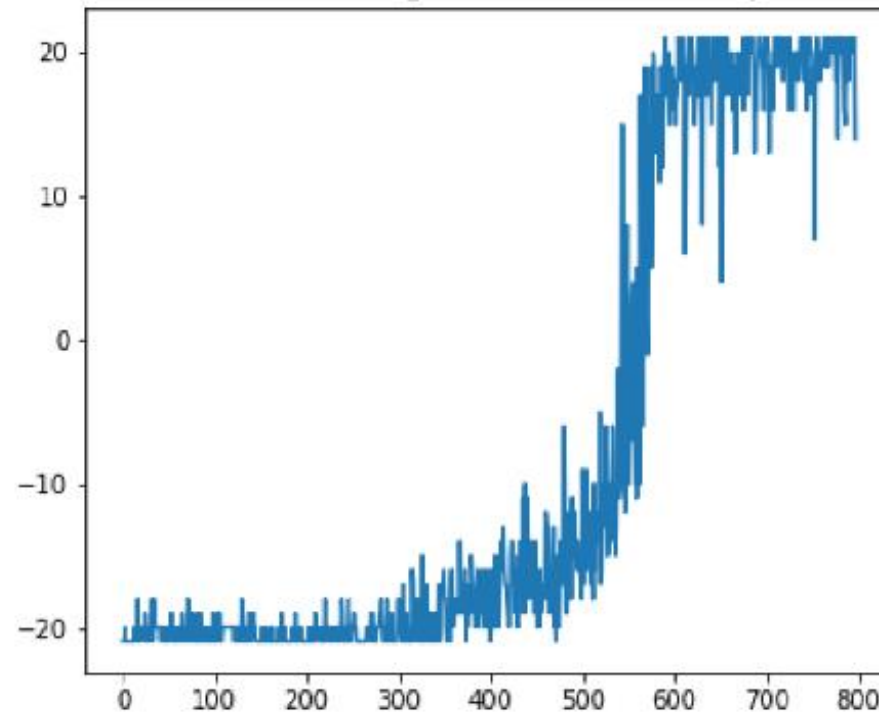
Source: Mnih et al. 2013



Source: Mnih et al. 2013

# Our Results (Pong)

Total frames 1440000. Avg reward over last 10 episodes: 19.6



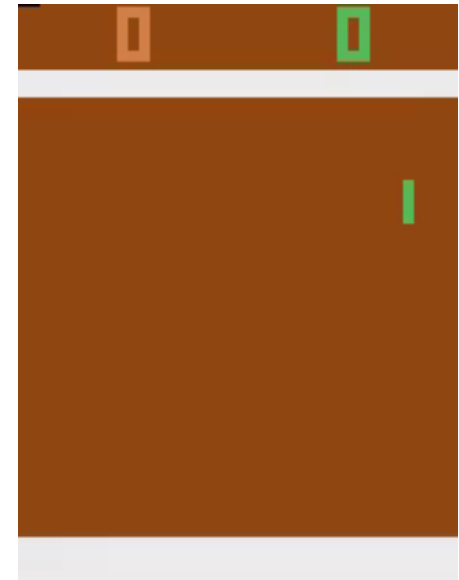
# Our Results (Pong)



400 episodes



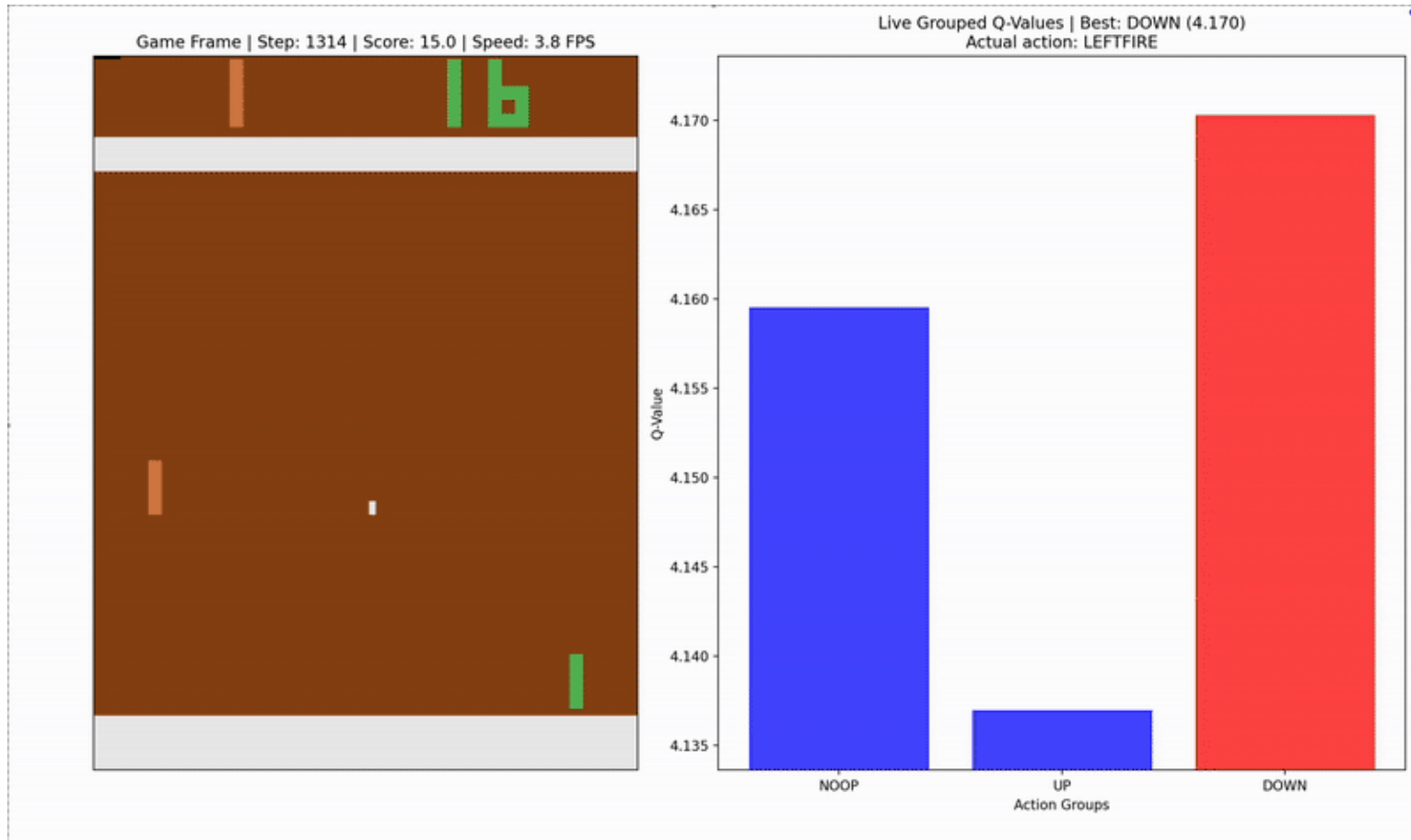
600 episodes



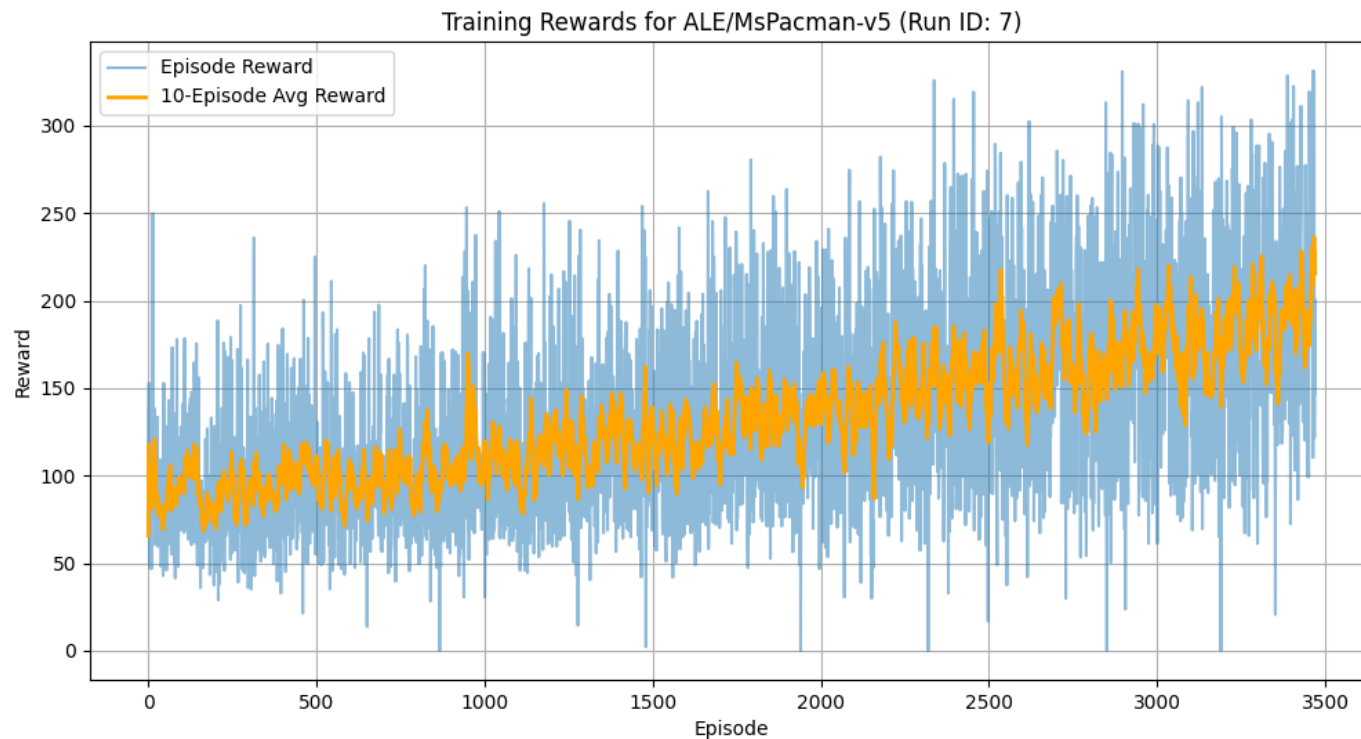
800 episodes



# Our Results (Pong)

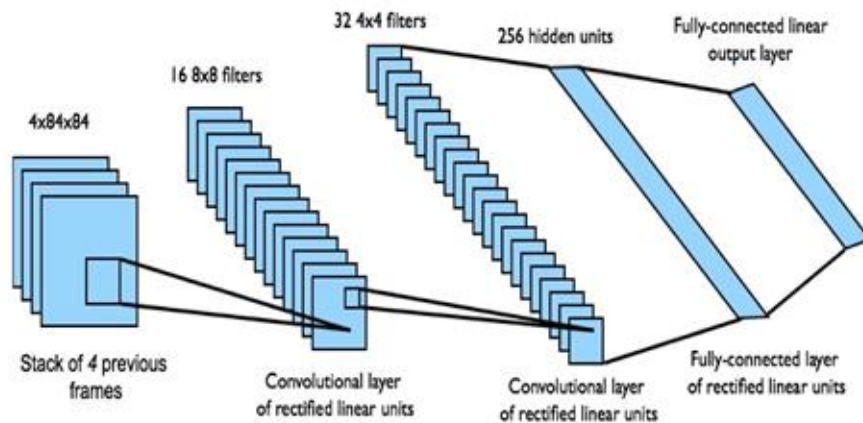


# Our Results (PacMan)

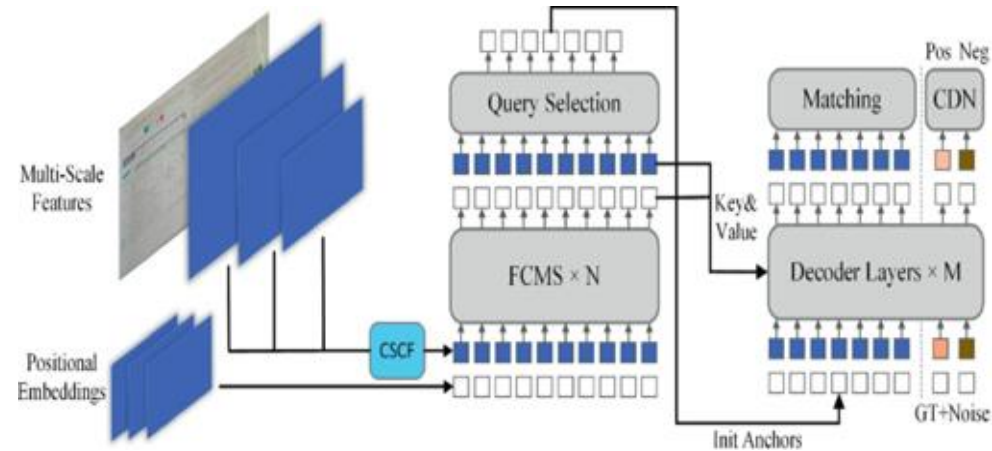


# Limitations & Subsequent Work

# Network Layout and Size



Convolutional Neural Network  
( $6 \cdot 10^5$  Parameters)

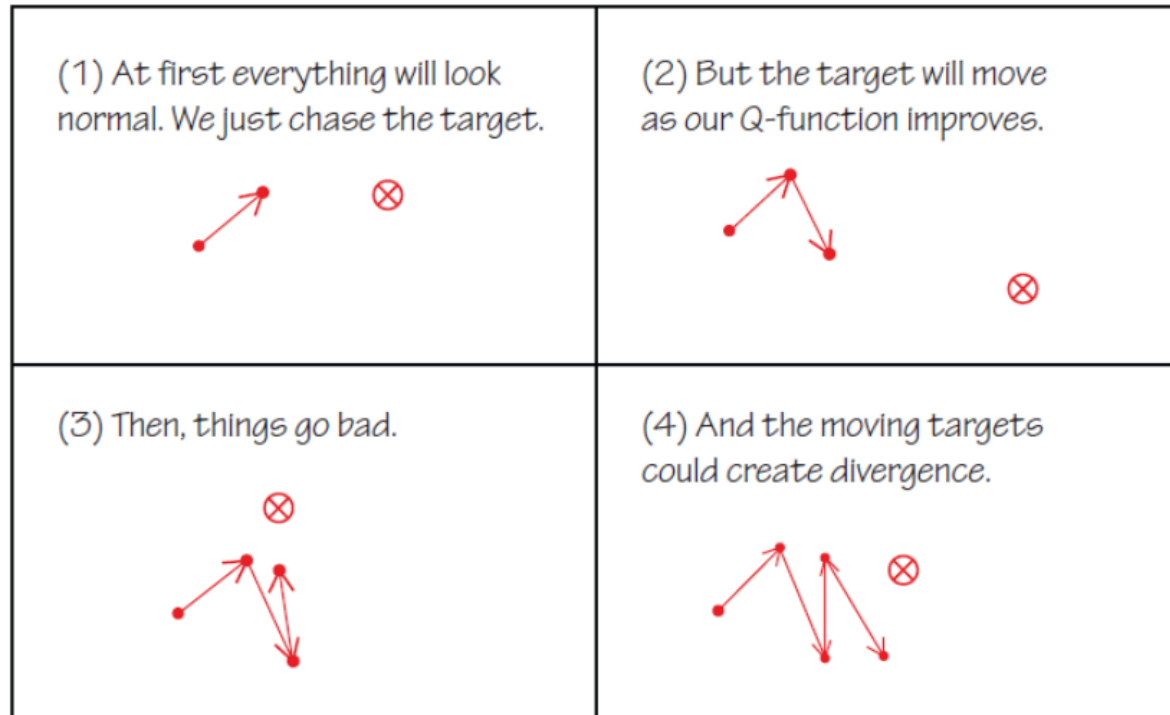


vs.

Modern Image Classifier  
(Up to  $8 \cdot 10^6$  Parameters)

# Loss/Reward Calculation

$$y_j = \begin{cases} r_j \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) \end{cases}$$



# Limitations



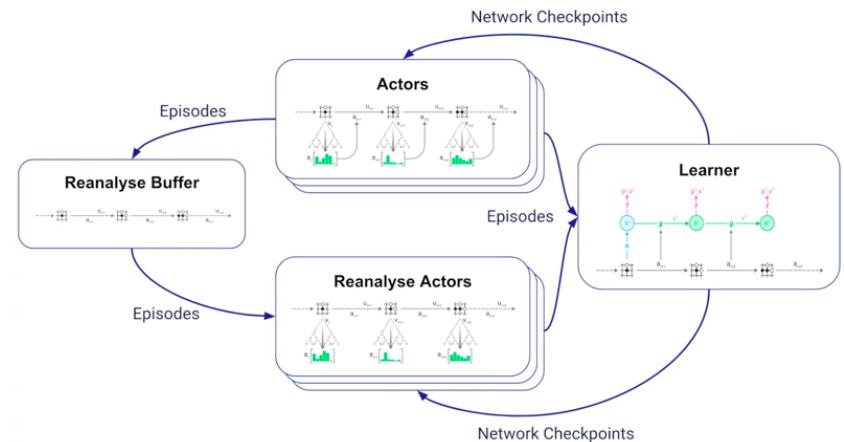
Montezumas Revenge

# Subsequent Work (DeepMind)



## AlphaGo/AlphaZero

First Computer Program to beat  
the Go World Champion  
(different architecture)



## MuZero

Current state-of-the-art for  
Reinforcement Learning e.g. on  
Atari Games

# Questions?

